Exercise Sheet 07

The Armoury

House rules.

- One file per exercise, in `my_work/`. Exercise 1 is done in the terminal, no file.
- Run everything **from the lesson folder**, with the virtual environment active (your prompt starts with `(.venv)`).
- Type the **Given** lines first, then write the rest. Compare with the **expected output**.
- Hand in: commit and push to `lesson-07`, open a pull request. Check `git status` first: `.venv/` must NOT be in it.

Exercise 1 An armoury of your own

10 min · terminal only

> **Raven from Orrin of the Clockworks:** *"Before you install anything: a virtual environment. It's a private armoury for this project, so whatever you forge here can't break anything else in the realm. Also I don't have to hear about it."*

IN THE TERMINAL, INSIDE THE LESSON FOLDER

```
python -m venv .venv            # build the armoury (once per project)
.venv\Scripts\activate          # open it: the prompt now starts with (.venv)
pip install -r requirements.txt # install what's on the list
pip list                        # what's in the armoury?
python -c "import rich; print('rich is ready')"
```

(a) Run the five commands. `pip list` shows `rich` and a few friends it brought along. Version numbers will differ from your neighbour's; that's fine.

(b) Open `requirements.txt` in VS Code and read it. That one line is the whole shopping list.

(c) Run `git status`. If `.venv/` shows up as untracked, tell Lars: the `.gitignore` should hide it.

EXPECTED OUTPUT (LAST COMMAND)

```
rich is ready
```

Hint: "Running scripts is disabled"? Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again. New terminal = activate again. VS Code usually does it for you once it has noticed the `.venv` folder.

Exercise 2 Borrow from the standard library

8 min · `ex2_maths.py`

> **Decree from Queen Aldara:** *"Lady Siv says I'm not allowed to say 'average' any more without saying which one. Also, if I split eight guards into patrols of three, how many patrols is that? Orrin laughed."*

GIVEN

```
import math
import statistics

salaries = [24950, 26550, 29000, 31750, 37650, 41900, 45200, 68750]
```

(a) Print the mean (2 decimals), the median, the standard deviation (`statistics.stdev`, 2 decimals) and the range (max minus min).

(b) Print how many patrols of 3 you need for 8 guards (`math.ceil`) and how many *full* patrols you get (`math.floor`).

(c) Print the total salary bill rounded *up* to the nearest 1,000 crowns, and the square root of 1019 to 3 decimals.

EXPECTED OUTPUT

```
Mean:    38218.75
Median:  34700.0
Std dev: 14311.93
Range:   43800
Patrols of 3: 3
Full patrols of 3: 2
Budget rounded up to the nearest 1,000 cr: 306000
Square root of 1019: 31.922
```

Hint: Nobody memorises the standard library. Type `python`, then `import statistics`, then `help(statistics.stdev)`. `q` gets you out of the help page. "Round up to the nearest 1,000" is `ceil(x / 1000) * 1000`.

Exercise 3 Dates without tears

12 min · `ex3_dates.py`

> **Raven from Mistress Pell (Paymaster):** *"Length of service, in years, as of the snapshot date, 5 April. In Excel this is a DATEDIF and a prayer. And when's payday? It's the 25th. I know when payday is. I want the script to know."*

GIVEN

```
from datetime import date, timedelta

today = date(2026, 4, 5)
start_dates = {"Pell": "2017-03-13", "Orrin": "2021-01-04", "Siv": "2024-11-28"}
```

(a) For each member of the court: turn the text into a date with `date.fromisoformat()`, work out the days since then (`(today - start).days`) and the years (divide by 365.25). Print the name in 8, the start date as "13 Mar 2017" and the weekday in brackets, then the years with 1 decimal.

(b) Payday is the 25th. Build this month's payday; if it's already gone, use next month's. Print it in ISO format and how many days away it is.

(c) The Accord report deadline is 90 days from today. Print it as dd/mm/yyyy.

EXPECTED OUTPUT

```
Pell     started 13 Mar 2017 (Monday), 9.1 years ago
Orrin    started 04 Jan 2021 (Monday), 5.2 years ago
Siv      started 28 Nov 2024 (Thursday), 1.3 years ago
Next payday: 2026-04-25 in 20 days
Accord report deadline: 04/07/2026
```

Hint: Dates format inside f-strings: `f"{start:%d %b %Y}"`, `f"{start:%A}"`. Codes: `%d` day, `%m` month number, `%b` short month name, `%Y` year, `%A` weekday. `today + timedelta(days=90)` does the calendar maths for you.

Exercise 4 Your own module

12 min · `paytools.py` + `ex4_use_paytools.py`

> **Raven from Orrin:** *"You've now typed `monthly salary = annual / 12` into six different files. The seventh time, forge it into a module and import it. Future Jen will send a thank-you raven."*

GIVEN (IN `MY_WORK/PAYTOOLS.PY`)

```
"""Pay calculations for the Royal Treasury of Thornmere. Import me, don't copy me."""
```

(a) In `paytools.py`, write four functions with docstrings: `monthly(annual_salary)` (2 decimals), `hourly(annual_salary, weekly_hours=37.5)` (52 weeks, 2 decimals), `pay_gap_percent(male_hourly, female_hourly)` (Accord definition, same as the UK's, 1 decimal) and `money(amount)` (crowns: commas, 2 decimals, `" cr"` on the end). No `print` anywhere in this file.

(b) In `ex4_use_paytools.py`: `import paytools`, then for Pell (32500), Orrin (41900) and Siv (37650) print the name in 8, the monthly pay (right-aligned in 12) and the hourly rate, both as money. Then print Orrin's hourly rate at 22.5 hours, and the gap for 20.00 cr vs 17.40 cr.

EXPECTED OUTPUT

```
Pell     2,708.33 cr a month, 16.67 cr an hour
Orrin    3,491.67 cr a month, 21.49 cr an hour
Siv      3,137.50 cr a month, 19.31 cr an hour
Part-time Orrin: 35.81
Gap: 13.0 %
```

Hint: Both files must be in `my_work/`, and you run `python my_work/ex4_use_paytools.py`: Python looks for modules next to the file it's running. In the REPL, `help(paytools)` shows your docstrings. That's what they're for.

Exercise 5 Biscuit's raffle

6 min · `ex5_raffle.py`

> **Raven from Lady Siv of the Rewards:** *"The Golden Quill of the Moon, the Midwinter raffle and the tea rota are all currently decided by the Queen 'at random'. Everyone has noticed it's always the Guard. Make it actually random. But the same random every time, so nobody can argue."*

GIVEN

```
import random

random.seed(1019)
names = ["Pell", "Orrin", "Siv", "Maren", "Bram", "Elowen", "Dunstan", "Freya"]
```

(a) Print one winner with `random.choice`, then three different raffle winners with `random.sample`, joined with ", ".

(b) `random.shuffle` the list and print the first four as a tea rota, joined with " -> ".

(c) Roll a die with `random.randint(1, 6)` for the length of Biscuit's flight.

EXPECTED OUTPUT

```
Golden quill of the moon goes to: Pell
Midwinter raffle winners: Elowen, Orrin, Pell
Tea rota: Pell -> Siv -> Dunstan -> Orrin
Dice for Biscuit's flight: 4
```

Hint: The seed makes "random" repeatable. Keep the calls in exactly this order, or you'll get different (equally random) names. Remove the seed line and run twice to see the difference.

Exercise 6 Something you didn't write

8 min · `ex6_rich_table.py`

> **Decree from Queen Aldara:** *"Can the terminal output look less like a tax stone? Orrin says there's 'a package for that'. There's always a package for that."*

GIVEN

```
from rich.console import Console
from rich.table import Table

roll = [("TM1092", "Nye Pennyworth", "Guard", 41900),
        ("TM1178", "Hild Woolpack", "Library", 26550),
        ("TM1105", "Maren Marchbank", "Guard", 24950)]
```

(a) Make a `Table(title="Roll of the Realm")` with columns ID, Name, Guild and Salary (right-justified). Add one row per subject, salary as crowns with commas.

(b) Print it with `Console(width=60).print(table)`, then a second line with "3 rows" in bold green: `"[bold green]3 rows[/bold green], no Grimoire involved"`.

EXPECTED OUTPUT (THE BOX LINES MAY LOOK SLIGHTLY DIFFERENT IN YOUR TERMINAL)

```
                Roll of the Realm
┏━━━━━━━━┳━━━━━━━━━━━━━━━━━━┳━━━━━━━━━┳━━━━━━━━━━━━┓
┃ ID     ┃ Name             ┃ Guild   ┃     Salary ┃
┡━━━━━━━━╇━━━━━━━━━━━━━━━━━━╇━━━━━━━━━╇━━━━━━━━━━━━┩
│ TM1092 │ Nye Pennyworth   │ Guard   │ 41,900 cr  │
│ TM1178 │ Hild Woolpack    │ Library │ 26,550 cr  │
│ TM1105 │ Maren Marchbank  │ Guard   │ 24,950 cr  │
└────────┴──────────────────┴─────────┴────────────┘
3 rows, no Grimoire involved
```

Hint: You've never seen `rich` before. That's the exercise: read the *Given*, guess from the names (`table.add_column(...)`, `table.add_row(...)`), and when stuck, search "rich table python". Reading other people's documentation is most of programming.

Exercise 7 The main guard STRETCH

8 min · `paytools.py` + `ex7_main_guard.py`

> **Raven from Orrin:** *"One more trick. A module should be quiet when imported and chatty when run on its own. There's a line for that. You've seen it before, in lesson 00, and nodded politely."*

GIVEN (ADD TO THE END OF `PAYTOOLS.PY`)

```
if __name__ == "__main__":
    print("paytools self-test")
```

(a) Under that line, print a one-line check for each of the four functions, in the format shown below.

(b) Run `python my_work/paytools.py`: you get the self-test.

(c) Write `ex7_main_guard.py`: `from paytools import monthly, money`, then print the two lines below. Run it: no self-test appears.

EXPECTED OUTPUT OF `PYTHON MY_WORK/PAYTOOLS.PY`

```
paytools self-test
monthly(32500)        -> 2708.33
hourly(32500)         -> 16.67
hourly(32500, 22.5)   -> 27.78
pay_gap_percent(20, 17.4) -> 13.0
money(2708.333)       -> 2,708.33 cr
```

EXPECTED OUTPUT OF `python my_work/ex7_main_guard.py`

```
Importing paytools printed nothing above this line.
Monthly pay for 45,200 cr: 3,766.67 cr
```

Hint: `__name__` is `"__main__"` only in the file you ran directly. Inside an imported module it's the module's name, so the block is skipped.

Bug hunt Pages from the Grimoire, page 7

15 min · `bughunt/toolbox_bug.py`

Dalvric's patrol planner for the Royal Guard, written the week he discovered `import`. There are **three** bugs: one crashes on the very first line, one rounds the wrong way so one patrol never gets a captain, and one prints something technically true but not what was asked.

EXPECTED OUTPUT ONCE FIXED

```
Guards: 9
Average salary: 31,644.44 cr
Patrols of 4 needed: 3
Days since Dalvric sailed west: 367653
```

Commit each fix separately, with a message explaining what was wrong.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.